

TT901A

Projeto Computacional - Primeira Etapa - Compressão de Fonte - JPEG

09 de Abril de 2026

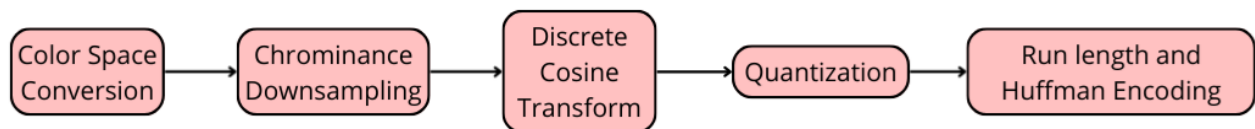
Em um sistema de transmissão digital de sinais, a informação da fonte passa por muitas etapas além de ser transformada em bits. Cada etapa da transmissão tem seu papel para garantir que o sinal transmitido pelo canal de comunicação (ar, fibra óptica, cabo coaxial) possa ser recuperado no receptor. Essa recuperação é necessária porque o canal pode distorcer o sinal de várias formas, tais como a partir da perda de potência, mudança de fase e interferência das ondas eletromagnéticas devido aos múltiplos percursos que implicam na chegada de sinais com diferentes atrasos no receptor. Sendo assim, podemos separar estas etapas de um sistema de transmissão digital da seguinte forma:

1. compressão da fonte: para que seja possível enviar mais dados utilizando menos banda;
2. codificação de canal: para que grande parte dos erros introduzidos pelo canal consigam ser corrigidos no receptor do sinal;
3. modulação e equalização: para possibilitar a transmissão em altas frequências (banda passante), aumentar a eficiência espectral e a taxas de dados, possibilitar a transmissão para vários canais em portadoras diferentes e para melhor a razão sinal-ruído. Além disso, a equalização minimiza a distorção introduzida pelo canal.

Todas as etapas são revertidas na ordem inversa após o sinal chegar ao receptor. Idealmente, o sinal seria recuperado exatamente como foi enviado, em um processo de transmissão denominado como "sem perdas". Na prática, várias etapas do processo de transmissão introduzem "ruído" não recuperável no sinal original, tal como o próprio processo de quantização, e o sinal original não pode ser recuperado exatamente como enviado. Para a grande maioria dos cenários de transmissão, isso não é um problema e as perdas são aceitáveis, muitas vezes sequer perceptíveis. Sendo assim, para determinar o nível de aceitação destas perdas de qualidade do sinal transmitido, são calculadas métricas de qualidade objetiva e subjetiva.

Neste projeto, implementaremos um sistema de transmissão digital de sinais para o envio de imagens e nosso projeto será dividido conforme as 3 etapas acima. Sendo assim, nesta primeira etapa, faremos a compressão da fonte a partir da implementação de uma versão simplificada do codec conhecido como JPEG (*Joint Photographic Experts Group*). A figura abaixo mostra um esquema resumido dos passos do codec, que serão cobertos na próxima seção. Para o melhor entendimento antes de se iniciar a implementação computacional, recomenda-se a leitura deste artigo simples e direto:

Al-Ani, Muzhir Shaban, and Fouad Hammadi Awad. "The JPEG image compression algorithm." *Int. J. Adv. Eng. Technol* 6.3 (2013): 1055-1062.



Atividades

Implementaremos primeiro a codificação, em seguida a decodificação e, finalmente, as métricas de avaliação de qualidade.

Parte 1: Codificação

[1] Carregamento e leitura da imagem

Ler uma imagem no formato .BMP e convertê-la para uma matriz de dimensões Altura x Comprimento x Canais para representar a imagem no formato RGB (*Red, Green, Blue*), onde cada canal representa uma das

cores. A variável que irá armazenar sua mensagem deve ser do tipo *ndarray*. **Dica:** use a biblioteca OpenCV do Python.

Exemplo de script para ler uma imagem pelo OpenCV no Python

```
import cv2

def ler_imagem(imagem):
    imagem_bgr = cv2.imread('caminho_para_a_imagem.bmp')
    imagem_rgb = cv2.cvtColor(imagem_bgr, cv2.COLOR_BGR2RGB)
    plt.imshow(imagem_rgb)
    plt.show()
```

[2] Conversão de RGB para YCbCr

Converter a imagem do formato RGB para o formato YCbCr, separando cada canal em uma variável diferente (faremos isso para facilitar a implementação, mas poderíamos manter como uma matriz de 3 dimensões). As variáveis que irão armazenar cada canal serão do tipo *ndarray*.

Dica: busque pelos pesos de conversão do formato RGB para o formato YCbCr e veja as fórmulas da conversão. Tente usar as funções da biblioteca *numpy* para fazer operações matriciais e evite o uso de laços *for*.

[3] Subamostragem da cromaticidade

Fazer a subamostragem da cromaticidade, uma técnica que se utiliza dos pontos fortes e fracos da visão humana e diminui a informação dos canais Cb e Cr. Use o formato 4:2:0 fazendo a média de um bloco com 4 pixels resultando em um novo bloco formado por apenas 1 pixel. As novas variáveis que irão armazenar cada canal subamostrado serão do tipo *ndarray*.

A subamostragem da cromaticidade tem três formatos típicos:

- 4:4:4 - imagem sem subamostragem, carregando toda informação de luminância e cor.
- 4:2:2 - possui metade da cromaticidade do formato 4:4:4. Cada 4 pixels de uma matriz de cromaticidade se tornam 2 pixels.
- 4:2:0 - possui um quarto da cromaticidade do formato 4:4:4. Cada 4 pixels de uma matriz de cromaticidade se tornam 1 pixel.

Dica: No item 2, separamos cada um dos canais (Y, Cb e Cr) para facilitar a implementação e isso será útil aqui. A subamostragem pode ser feita com laços *for*, mas é possível fazer de formas mais eficientes usando o *numpy*.

[4] Conversão para blocos

Converter cada um dos canais (Y, Cb e Cr) em blocos 8x8, preparando a base para a aplicação da *Discrete Cosine Transform* (DCT). Após isso, juntá-los em uma mesma variável.

Imagens em geral possuem uma grande correlação espacial local, ou seja, se selecionarmos um pedaço da imagem é provável que um pixel tenha alta correlação com os pixels adjacentes. Além disso, o poder computacional necessário para realizar operações matriciais em vários blocos de tamanho fixo N (nesse caso, $N = 8$) é menor que o necessário para fazer o mesmo em uma imagem de tamanho 512x512, por exemplo.

Dica: a entrada da sua função para criar os blocos será um *ndarray*, mas para que seja mais fácil lidar com os passos seguintes a saída deve ser uma lista de *ndarrays* (onde cada *ndarray* é um bloco 8x8). Para isso, faça com que sua função crie uma lista vazia e aproveite o método *.append()* das listas do Python. Ao fim, você terá 3 listas Python, uma para cada canal. Basta então juntá-las na ordem correta (Y, Cb, Cr) utilizando as propriedades de listas Python.

[5] Matriz da DCT

Compute os coeficientes da matriz da *Discrete Cosine Transform* (DCT) para um bloco $N \times N$.

A DCT possui uma relação grande com a *Discrete Fourier Transform* (DFT). A DCT trabalha com números reais, usando cossenos e assumindo a simetria ímpar do sinal, enquanto a DFT trabalha com senóides complexas.

A DCT é melhor para compressão de imagens pois consegue concentrar a energia do sinal em menos coeficientes de baixa frequência, evitando os artefatos de alta frequência causados pelas descontinuidades que aparecem na periodicidade da DFT.

A equação que computa a i,j -ésima entrada da DCT de uma imagem é:

$$D(i, j) = \frac{1}{\sqrt{2N}} C(i) C(j) \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} p(x, y) \cos \left[\frac{(2x+1)i\pi}{2N} \right] \cos \left[\frac{(2y+1)j\pi}{2N} \right], \quad (1)$$

onde

$$C(u) = \begin{cases} \frac{1}{\sqrt{2}} & \text{if } u = 0 \\ 1 & \text{if } u > 0 \end{cases}$$

Na Equação (1), $p(x, y)$ é o elemento (x, y) da imagem representada pela matriz p . N é o tamanho do bloco sobre o qual a DCT é aplicada. A equação calcula uma entrada (i, j) da imagem transformada a partir dos valores de pixel da matriz da imagem original. Para o bloco padrão de 8×8 que a compressão JPEG utiliza, N é igual a 8, e x e y variam de 0 a 7. Dessa forma, a Equação (1), considerando um bloco de tamanho $N = 8$ é:

$$D(i, j) = \frac{1}{4} C(i) C(j) \sum_{x=0}^7 \sum_{y=0}^7 p(x, y) \cos \left[\frac{(2x+1)i\pi}{16} \right] \cos \left[\frac{(2y+1)j\pi}{16} \right] \quad (2)$$

Para obter a forma matricial da Equação (1), temos que:

$$T_{i,j} = \begin{cases} \frac{1}{\sqrt{N}} & \text{if } i = 0 \\ \sqrt{\frac{2}{N}} \cos \left[\frac{(2j+1)i\pi}{2N} \right] & \text{if } i > 0 \end{cases} \quad (3)$$

E para um bloco 8×8 , isso resulta em:

$$T = \begin{bmatrix} .3536 & .3536 & .3536 & .3536 & .3536 & .3536 & .3536 & .3536 \\ .4904 & .4157 & .2778 & .0975 & -.0975 & -.2778 & -.4157 & -.4904 \\ .4619 & .1913 & -.1913 & -.4619 & -.4619 & -.1913 & .1913 & .4619 \\ .4157 & -.0975 & -.4904 & -.2778 & .2778 & .4904 & .0975 & -.4157 \\ .3536 & -.3536 & -.3536 & .3536 & .3536 & -.3536 & -.3536 & .3536 \\ .2778 & -.4904 & .0975 & .4157 & -.4157 & -.0975 & .4904 & -.2778 \\ .1913 & -.4619 & .4619 & -.1913 & -.1913 & .4619 & -.4619 & .1913 \\ .0975 & -.2778 & .4157 & -.4904 & .4904 & -.4157 & .2778 & -.0975 \end{bmatrix} \quad (4)$$

Note que a primeira linha da matriz (para $i = 0$) todos resultados são iguais a $\frac{1}{\sqrt{8}}$, como era esperado de acordo com a Equação (4). **A matriz que você computar deve gerar os mesmos coeficientes que a matriz mostrada na Equação (4).**

Dica: a ideia desse item é a implementação da equação matricial da DCT em código, mas você só vai precisar fazer esse cálculo uma vez no código. Por isso, faça uma função separada que só vai ser executada uma vez. A saída deve ser um `ndarray`.

[6] Aplicação da DCT

Aplicar a DCT (*Discrete Cosine Transform*) em cada um dos canais.

Uma vez que a matriz da DCT foi calculada, podemos usar a equação de matrizes que aplica a DCT em um bloco 8×8 que contém pixels da imagem, mas primeiro precisamos normalizar esses pixels pelo fato de que a DCT foi pensada para funcionar com valores no intervalo de -128 a 127 e, por padrão, intervalo de valores de um pixel varia de 0 a 255. No exemplo a seguir, a Equação (5) representa um bloco de pixels e a Equação (6) mostra esse mesmo bloco após a normalização.

$$Original = \begin{bmatrix} 154 & 123 & 123 & 123 & 123 & 123 & 123 & 136 \\ 192 & 180 & 136 & 154 & 154 & 154 & 136 & 110 \\ 254 & 198 & 154 & 154 & 180 & 154 & 123 & 123 \\ 239 & 180 & 136 & 180 & 180 & 166 & 123 & 123 \\ 180 & 154 & 136 & 167 & 166 & 149 & 136 & 136 \\ 128 & 136 & 123 & 136 & 154 & 180 & 198 & 154 \\ 123 & 105 & 110 & 149 & 136 & 136 & 180 & 166 \\ 110 & 136 & 123 & 123 & 123 & 136 & 154 & 136 \end{bmatrix} \quad (5)$$

$$M = \begin{bmatrix} 26 & -5 & -5 & -5 & -5 & -5 & -5 & 8 \\ 64 & 52 & 8 & 26 & 26 & 26 & 8 & -18 \\ 126 & 70 & 26 & 26 & 52 & 26 & -5 & -5 \\ 111 & 52 & 8 & 52 & 52 & 38 & -5 & -5 \\ 52 & 26 & 8 & 39 & 38 & 21 & 8 & 8 \\ 0 & 8 & -5 & 8 & 26 & 52 & 70 & 26 \\ -5 & -23 & -18 & 21 & 8 & 8 & 52 & 38 \\ -18 & 8 & -5 & -5 & -5 & 8 & 26 & 8 \end{bmatrix} \quad (6)$$

Com isso definido, podemos realizar a DCT por multiplicação de matrizes como mostra a Equação (7):

$$D = TMT' \quad (7)$$

sendo que todas, como mostrado nos passos anteriores, são matrizes 8x8, assim garantindo que as dimensões estejam corretas para a multiplicação.

Para verificar se suas funções estão funcionando devidamente, faça o teste com os valores das matrizes T e M vistas nesse passo e nos passos anteriores. O resultado deve ser de acordo com a Equação (8) (podem haver pequenas diferenças devido a arredondamentos):

$$D = \begin{bmatrix} 162.25 & 40.60 & 20.00 & 72.33 & 30.25 & 12.48 & -19.65 & -11.50 \\ 30.48 & 108.42 & 10.47 & 32.29 & 27.70 & -15.50 & 18.41 & -2.00 \\ -94.14 & -60.05 & 12.30 & -43.42 & -31.29 & 6.07 & -3.33 & 7.14 \\ -38.57 & -83.36 & -5.41 & -22.17 & -13.52 & 15.49 & -1.33 & 3.53 \\ -31.25 & 17.93 & -5.52 & -12.36 & 14.25 & -5.96 & 11.49 & -6.02 \\ -0.86 & -11.76 & 12.78 & 0.18 & 28.07 & 12.57 & 8.35 & 2.94 \\ 4.63 & -2.42 & 12.17 & 6.56 & -18.70 & -12.75 & 7.70 & 12.03 \\ -9.95 & 11.19 & 7.81 & -16.29 & 21.46 & 0.02 & 5.91 & 10.68 \end{bmatrix} \quad (8)$$

Dica: essa é a primeira parte da implementação na qual você terá que lidar com a lista de blocos criada anteriormente. Usando um laço for, é possível iterar por cada um dos blocos (que são *ndarrays*) que estão nessa lista e fazer a normalização nos pixels para então obter o resultado da equação $D = TMT'$. Para manter o mesmo padrão de implementação, lembre-se da dica dada no item 4.

[7] Quantização

Fazer a quantização dos blocos a partir da matriz de quantização Q_{50} fornecida no script "utils.py". Use "import utils" no começo do seu código para poder utilizar as funções desse script.

A quantização é um processo essencial para comprimir a informação de forma a representar os coeficientes da DCT com precisão suficiente para obter a qualidade desejada de imagem, ou seja, descartar informações que não terão impacto visual. É importante lembrar que a quantização é um processo com perdas, não sendo possível recuperar exatamente a informação que tínhamos antes dela.

A quantização, nesse caso, é dada pela divisão ponto-a-ponto da equação (9):

$$D_q = \left\lfloor \frac{D}{Q_{50}} \right\rfloor, \quad (9)$$

onde D_q é o bloco com os coeficientes quantizados e $\lfloor \cdot \rfloor$ representa o arredondamento para o inteiro mais próximo.

No JPEG, podemos escolher o nível de compressão da imagem nessa etapa a partir de matrizes de quantização associadas a um nível que vai de 1 a 100, onde 1 fornece a melhor compressão mas a menor qualidade. As Equações (10), (11) e (12) mostram três exemplos de matrizes de quantização do JPEG.

$$Q_{10} = \begin{bmatrix} 80 & 60 & 50 & 80 & 120 & 200 & 255 & 255 \\ 55 & 60 & 70 & 95 & 130 & 255 & 255 & 255 \\ 70 & 65 & 80 & 120 & 200 & 255 & 255 & 255 \\ 70 & 85 & 110 & 145 & 255 & 255 & 255 & 255 \\ 90 & 110 & 185 & 255 & 255 & 255 & 255 & 255 \\ 120 & 175 & 255 & 255 & 255 & 255 & 255 & 255 \\ 245 & 255 & 255 & 255 & 255 & 255 & 255 & 255 \\ 255 & 255 & 255 & 255 & 255 & 255 & 255 & 255 \end{bmatrix} \quad (10)$$

$$Q_{50} = \begin{bmatrix} 1 & 1 & 2 & 4 & 8 & 16 & 32 & 64 \\ 1 & 1 & 2 & 4 & 8 & 16 & 32 & 64 \\ 2 & 2 & 2 & 4 & 8 & 16 & 32 & 64 \\ 4 & 4 & 4 & 4 & 8 & 16 & 32 & 64 \\ 8 & 8 & 8 & 8 & 8 & 16 & 32 & 64 \\ 16 & 16 & 16 & 16 & 16 & 16 & 32 & 64 \\ 32 & 32 & 32 & 32 & 32 & 32 & 32 & 64 \\ 64 & 64 & 64 & 64 & 64 & 64 & 64 & 64 \end{bmatrix} \quad (11)$$

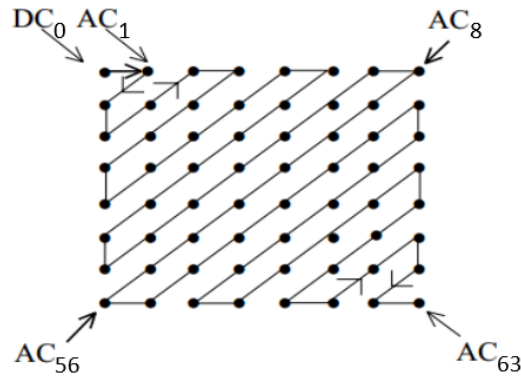
$$Q_{90} = \begin{bmatrix} 3 & 2 & 2 & 3 & 5 & 8 & 10 & 12 \\ 2 & 2 & 3 & 4 & 5 & 12 & 12 & 11 \\ 3 & 3 & 3 & 5 & 8 & 11 & 14 & 11 \\ 3 & 3 & 4 & 6 & 10 & 17 & 16 & 12 \\ 4 & 4 & 7 & 11 & 14 & 22 & 21 & 15 \\ 5 & 7 & 11 & 13 & 16 & 12 & 23 & 18 \\ 10 & 13 & 16 & 17 & 21 & 24 & 24 & 21 \\ 14 & 18 & 19 & 20 & 22 & 20 & 20 & 20 \end{bmatrix} \quad (12)$$

[8] Differential Pulse Code Modulation (DPCM)

Realizar a DPCM no coeficiente da posição (0,0) de todos os blocos da DCT.

No contexto da DCT aplicada a um bloco 8x8, o coeficiente da posição (0,0) é a média de todos os 64 pixels do bloco. A partir disso, ele foi nomeado de componente DC (*Direct Current*) como analogia a corrente contínua e os 63 coeficientes restantes foram então chamados de AC (*Alternate Current*).

Dessa forma, foi criada uma separação entre eles, explorando o fato de que os componentes DC de blocos vizinhos tendem a ser parecidos. É por isso que em vez de guardar o valor absoluto, um componente DC guarda a diferença em relação ao bloco anterior. A Figura a seguir mostra os componentes DC e AC.



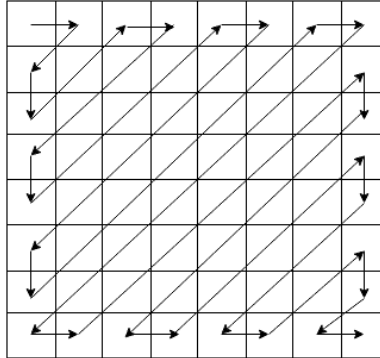
Dica: a implementação dessa função exige muita atenção, pois ela deve ser aplicada separadamente em cada canal. Lembre-se que as listas de blocos correspondentes aos canais Y, Cb e Cr foram unidos, então você precisará manipulá-los de forma separada ao aplicar a DPCM nos componentes DC dos blocos.

[9] Ordenação em zigue-zague

Reordene os valores do *ndarray* com o método de zigue-zague.

Após a quantização, é comum que vários coeficientes sejam iguais ou próximos a zero. Tipicamente, os coeficientes de baixa frequência (para os quais nossos olhos são mais sensíveis) são os de valores mais altos em imagens naturais.

Utilizando-se dessas informações, o método em zigue-zague ordena primeiro as frequências mais baixas e depois as mais altas. A maioria dos zeros serão agrupados e isso terá grande utilidade a seguir. Essa ordenação transformará a matriz 8x8 em um vetor de 64 elementos. A Figura a seguir mostra detalhadamente a sequência de ordenação do zigue-zague no bloco.



Dica: mantenha a lógica da lista de *ndarrays*. A única diferença nesse momento será que teremos a matriz 8x8 "linearizada" em um vetor de 64 elementos.

[10] Codificação de corrida de zeros

Como apontado anteriormente, depois de aplicar a ordenação em zigue-zague, o vetor resultante provavelmente contará com vários zeros em sequência. A codificação de corrida de zeros vai agrupar esses zeros e diminuir o número de elementos que precisaremos codificar com o código de Huffman na próxima etapa. A Equação (13) exemplifica essa codificação:

$$[10, 5, 20, 30, 0, 0, 0, 0, 2, 0, 0, 0] \longrightarrow [(0, 10), (0, 5), (0, 20), (0, 30), (4, 2), \text{EOB}], \quad (13)$$

onde EOB é o *End Of Block* (fim do bloco), indicando que a partir dele todos os valores serão zero.

Atenção: implemente seu código de forma que o resultado continue sendo uma lista de blocos, mas que cada bloco seja composto pelas tuplas obtidas (assim como mostra a Equação (15)). Será necessário que seu código esteja dessa exata forma para que a função do próximo item [11] funcione.

[11] Codificação de Huffman

Aplicar a codificação de Huffman e obter a *bitstream* do final da etapa de codificação do JPEG.

Use a função `encode_huffman` do arquivo `huffman.py` para fazer a codificação dos valores em bits (é necessário usar `"import huffman"` para ter acesso a ela).

Dica: esta é a parte mais complexa da implementação do JPEG, por isso a estamos fornecendo no arquivo a ser importando. Mesmo assim, é importante compreendê-la e para isso, recomenda-se a leitura do artigo:

Huffman, D.A. A method for the construction of minimum redundancy codes, *Proceedings IRE*, vol. 40, pp 1098-1101, 1962.

[12] Métricas de compressão

Finalizada a etapa da codificação, vamos calcular algumas métricas de compressão! Crie uma função que imprima as seguintes informações: tamanho da imagem original em bits, tamanho da *bitstream* resultante da codificação de Huffman, razão de compressão e bits por pixel.

A quantidade original de bits da imagem em .bmp é calculada considerando as dimensões da imagem, o número de canais e que temos 8 bits por canal. A Equação (13) mostra um exemplo onde a imagem tem dimensões 512x512 e é RGB.

$$O = 512 \cdot 512 \cdot 3 \cdot 8 = 6,291,456 \text{ bits} \quad (14)$$

A razão de compressão é obtida a partir da relação entre o tamanho original da imagem (em bits) com o tamanho da sequência codificada de bits obtida por Huffman.

Os bits por pixel (BPP), por outro lado, são calculados a partir da razão entre o tamanho da sequência codificada de bits pelo tamanho da imagem. Lembre-se que a imagem foi comprimida nessa sequência de bits, então os 3 canais já estão embutidos nela.

Parte 2: Decodificação

Agora vamos reverter, na ordem inversa, todos os passos feitos na codificação!

[1] Decodificação de Huffman

Use a função `decode_huffman.py` do arquivo `huffman.py` para fazer a decodificação dos bits em uma lista de blocos compostos por tuplas novamente.

[2] Decodificação de corrida de zeros

Reverta a diminuição de valores criada por essa codificação fazendo o processo contrário, os transformando em um `ndarray` de valores inteiros.

[3] Ordenação inversa em zigue-zague

Reordene os elementos como estavam originalmente, sendo que será necessário separar os 64 elementos (dado que os blocos da DCT são 8x8), aplicar a reordenação e transformar o vetor de 64 elementos em uma matriz 8x8 novamente.

Dica: faça esses processos na mesma função, mas em laços `for` diferentes. A sua implementação da função original de ordenação zigue-zague pode ter informações úteis para a reversão dela.

[4] DPCM Inversa

Inverta o processo da DPCM reconstruindo os valores DC originais, novamente fazendo o processo para cada canal de forma separada.

[5] Dequantização

Reverta a quantização. Esse processo é simples, basta usar a operação oposta usada na quantização, como mostra a Equação (15):

$$D_{deg} = D \cdot Q_{50} \quad (15)$$

[6] DCT Inversa

Aplique a DCT inversa para recuperar os canais Y, Cb e Cr. A *Inverse Discrete Cosine Transform* (IDCT) em sua forma matricial é muito similar à DCT e usa a matriz de coeficientes que já calculamos, como mostra a Equação (16).

$$D_{inv} = T' M T \quad (16)$$

[7] Separar os canais Y, Cb e Cr e reconstruir as matrizes

Separe os canais Y, Cb e Cr fatiando as listas Python e a partir disso reconstrua as matrizes originais dos canais. Com a IDCT aplicada, a lista de blocos volta a ser a união dos canais Y, Cb e Cr. Considerando o número de blocos que cada canal gerou, é possível "recriar" os três canais.

A partir dos blocos 8x8 de cada canal, é possível reconstruir as respectivas matrizes. Será necessário iterar por uma matriz vazia em passos de 8 de linhas e 8 colunas colocando cada bloco no seu devido lugar. Lembre-se que antes da DCT, os pixels foram normalizados para estarem no intervalo de -128 a 127, e será necessário que eles voltem a estar entre 0 e 255.

[8] Superamostragem de Crominância

Reverta a subamostragem de crominância (canais Cb e Cr). Ao contrário de antes, cada 1 pixel será replicado 4 vezes, formando um bloco 2x2 e eventualmente a matriz 256x256 voltará a ter o tamanho 512x512 e assim será possível ler e mostrá-la novamente.

[9] Converter YCbCr para RGB

Converter a imagem de YCbCr para RGB, finalmente reunindo os canais na mesma variável reconstruindo a matriz 512x512x3. Assim como na conversão de RGB para YCbCr, procure os pesos e as fórmulas para essa conversão. Leia e mostre a imagem reconstruída.

Parte 3: Métricas finais e estudo de parâmetros

[1] Métricas

Utilizaremos duas métricas de qualidade objetiva para a avaliação da imagem reconstruída: o PSNR (*Peak Signal-to-Noise Ratio* - Relação Sinal-Ruído de Pico) e o SSIM (*Structural Similarity Index Measure* - Índice de Similaridade Estrutural). Pode-se utilizar bibliotecas com essas funções prontas.

Compare a imagem original com a imagem reconstruída e mostre os resultados. Em uma análise considerando que o JPEG é um método bastante comum de compressão, qual o significado desses resultados? Imprima seus comentários através do código.

[2] Parâmetros e avaliação subjetiva

Agora vamos mudar a taxa de compressão e observar como a compressão se comporta!

Mude a tabela de quantização usada (Q50) para a tabela (Q10) e depois para a tabela (Q90), executando o código novamente a cada mudança.

Qual a diferença entre o uso dessas tabelas?

Existe alguma diferença visível nas imagens reconstruídas para cada quantização?

Qual é o uso prático de níveis de quantização na compressão do JPEG?

Imprima seus comentários respondendo a estas perguntas através do código.

Instruções para a entrega da atividade

Para esta atividade é esperado somente UM notebook Jupyter (arquivo .ipynb) que deve ser entregue no Classroom e utilizado para gerar os resultados.

Para cada etapa, crie uma célula de texto (*Markdown*) e uma célula de código correspondente. Em cada célula de código, crie uma função Python com a implementação da etapa e chame essa função para executá-la e obter o resultado da etapa.

Inclua todas as discussões, justificativas e comentários pedidos nas células de texto (*Markdown*) no próprio notebook Jupyter.

Entregas em formatos diferentes e/ou com atraso terão penalidades de 50% na nota total. Além disso, o uso grosseiro de LLMs também terá penalidade de 50% na nota. Lembrando que o uso de LLMs é incentivado como auxílio na realização do exercício, não como substituto completo.